

MANUAL DE INSTALACIÓN

KN-STORE

Proyecto: KN-Store

Versión: 1.0

Fecha: 20 de agosto de 2026

Elaborado por:

Laura González Ortiz

Santiago Flórez Moreno

Joseph Varón

Nicolás Gil

Tecnología: React, Spring Boot, MongoDB, Docker

Ambiente: Producción

Tabla de contenido

PARTE I.....	3
INFORMACIÓN GENERAL DEL PROYECTO.....	3
1. Introducción.....	3
2. Objetivo.....	3
3. Alcance.....	3
4. Requisitos del Sistema.....	3
5. Tecnologías Utilizadas	4
PARTE II INSTALACIÓN Y CONFIGURACIÓN DEL ENTORNO.....	5
6. Despliegue con Docker.....	5
6.1 Configuración de MongoDB	5
6.2 Configuración de la aplicación.....	6
6.3 Verificación de los contenedores.....	6
6.5 Detener los servicios.....	7
7. Configuración del servidor AWS	8
7.1 Creación de la instancia EC2	8
7.2 Conectarse mediante SSH	11
8. Configuración del DNS	13
9. Dominio y SSL.....	13
9.1 Configuración del Administrador de proxy Nginx	13
9.2 Acceso a Nginx Proxy Manager	14
9.3 Configuración del Proxy Host.....	14
9.4 Creación del Proxy Host.....	14
9.5 Configuración del certificado SSL.....	15
9.6 Configuración general de Nginx Proxy Manager	15
9.7 Resultado de la configuración.....	16
10. Resumen del Manual de Instalación	16
11. Conclusiones.....	17

PARTE I

INFORMACIÓN GENERAL DEL PROYECTO

1. Introducción

El presente Manual de Instalación describe el proceso necesario para instalar, configurar y ejecutar el sistema **KN-Store**, una plataforma web desarrollada para la gestión y comercialización de productos de una tienda de calzado.

En este documento se presentan los requisitos del sistema, las herramientas necesarias y los pasos para desplegar la aplicación en un servidor de producción, con el fin de garantizar una correcta implementación y funcionamiento de la aplicación.

2. Objetivo

Brindar una guía clara y organizada para la instalación, configuración y puesta en marcha del sistema **KN-Store**, permitiendo que desarrolladores, administradores o personal de soporte puedan preparar el entorno de trabajo y ejecutar la aplicación correctamente.

De igual manera, este manual busca facilitar futuras instalaciones, actualizaciones y tareas de mantenimiento del proyecto.

3. Alcance

Este manual está dirigido a los integrantes del equipo de desarrollo, administradores de sistemas y personal de soporte técnico encargado de la implementación del proyecto **KN-Store**.

El documento abarca el despliegue de la aplicación mediante Docker, la configuración del servidor en AWS, la preparación de la base de datos y la configuración del dominio y el certificado SSL para el ambiente de producción.

4. Requisitos del Sistema

Para ejecutar correctamente el proyecto **KN-Store**, el equipo debe cumplir con los siguientes requisitos mínimos:

Hardware

Recurso	Mínimo	Recomendado
Procesador	2 núcleos	4 núcleos
Memoria RAM	4 GB	8 GB
Almacenamiento	20 GB	50 GB SSD
Conexión a Internet	Requerida	Banda ancha estable

Sistema Operativo

El proyecto puede instalarse en los siguientes sistemas operativos:

- Windows 10 o superior.
- Ubuntu 22.04 LTS o superior.
- Debian 12 o superior.

Software requerido

Antes de iniciar el despliegue, el servidor de producción debe contar con Docker y Docker Compose correctamente instalados y configurados, ya que son las herramientas utilizadas para ejecutar la aplicación y sus servicios asociados (backend y MongoDB). Adicionalmente, la imagen de la aplicación (knstore:latest) debe estar previamente construida y disponible para su despliegue.

5. Tecnologías Utilizadas

El desarrollo de **KN-Store** se realizó utilizando diferentes tecnologías que permiten ofrecer una aplicación moderna, escalable y de fácil mantenimiento.

- **Java:** Lenguaje utilizado para el desarrollo del backend.
- **Spring Boot:** Framework que facilita la creación y gestión de servicios del servidor.
- **React:** Biblioteca utilizada para el desarrollo de la interfaz de usuario.
- **Node.js y npm:** Utilizados para ejecutar el entorno del frontend y administrar sus dependencias.
- **Apache Maven:** Herramienta encargada de la compilación y gestión de dependencias del backend.
- **MongoDB:** Base de datos NoSQL utilizada para almacenar la información del sistema.
- **Docker y Docker Compose:** Utilizados para contenerizar y desplegar la aplicación de manera sencilla y estandarizada.
- **Git:** Sistema de control de versiones empleado para el trabajo colaborativo y el seguimiento de los cambios realizados en el proyecto.

PARTE II

INSTALACIÓN Y CONFIGURACIÓN DEL ENTORNO

6. Despliegue con Docker

El proyecto **KN-Store** utiliza Docker para facilitar la ejecución de los servicios necesarios para el funcionamiento de la aplicación.

La configuración de Docker se encuentra en:

src/main/docker/

Dentro de esta carpeta se encuentran diferentes archivos de configuración .yml utilizados para administrar los servicios de la aplicación.

Para el despliegue de KN-Store se utilizan principalmente:

- **app.yml:** contiene la configuración del contenedor principal de la aplicación.
- **mongodb.yml:** contiene la configuración del servicio de base de datos MongoDB.

Docker permite ejecutar estos servicios de manera independiente y establecer la comunicación necesaria entre la aplicación y la base de datos.

6.1 Configuración de MongoDB

KN-Store utiliza **MongoDB** como sistema de gestión de base de datos. El servicio se encuentra definido en el archivo: **mongodb.yml**.

La base de datos se ejecuta dentro de un contenedor Docker y cuenta con un volumen denominado: **knstore-mongodb-data**

```
volumes:
  - knstore-mongodb-data:/data/db/
  - knstore-mongodb-data:/data/configdb/
```

Este volumen permite conservar la información almacenada en la base de datos, incluso cuando los contenedores sean detenidos o reiniciados.

```
mongodb:
  image: mongo:8.2.9
  # Replica set de un solo nodo para habilitar replicación
  command: ['mongod', '--replSet', 'rs0', '--storageEngine', 'wiredtiger']
  # If you want to expose these ports outside your host's network,
  # remove the "127.0.0.1:" prefix
  networks:
    - knstore
  ports:
    - :27018:27017
  volumes:
    - knstore-mongodb-data:/data/db/
    - knstore-mongodb-data:/data/configdb/
```

6.2 Configuración de la aplicación

El archivo **app.yml** contiene la configuración del contenedor principal de la aplicación KN-Store.

Este contenedor ejecuta los servicios necesarios para el funcionamiento del backend y establece comunicación con la base de datos MongoDB mediante la infraestructura de red interna administrada por Docker.

La imagen utilizada por la aplicación es:

image: **knstore:latest**

```
name: knstore
services:
  app:
    image: knstore:latest
    environment:
      - _JAVA_OPTIONS=-Xmx512m -Xms256m
      - SPRING_PROFILES_ACTIVE=dev,api-docs,secret-samples
      - MANAGEMENT_PROMETHEUS_METRICS_EXPORT_ENABLED=true
      - SPRING_MONGODB_URI=mongodb://mongodb:27017/knstore?
      - SPRING_DOCKER_COMPOSE_ENABLED=false
    ports:
      - 8080:8080
    networks:
      - knstore
```

La aplicación se encuentra desplegada en un servidor de producción, por lo que el acceso de los usuarios se realiza mediante el dominio configurado para KN-Store y no directamente mediante una dirección localhost.

6.3 Verificación de los contenedores

Una vez realizado el despliegue, es necesario verificar que los contenedores requeridos para el funcionamiento de KN-Store se encuentren activos.

Para ello, ejecute el siguiente comando:

docker ps

```
PS C:\Users\APRENDIZ\Documents\Proyecto_KN_S\2026-3311941-projects-grupo-06-knstore\src\main\docker> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
ea2caa261434	knstore:latest	"bash -c /entrypoint..."	9 minutes ago	Up About a minute (healthy)	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp	knstore-app-1
33bcb22811ee	mongo:8.2.9	"docker-entrypoint.s..."	2 hours ago	Up 2 hours (healthy)	0.0.0.0:27018->27017/tcp, [::]:27018->27017/tcp	knstore-mongodb-1

6.4 Acceso a la aplicación

Una vez verificado el correcto funcionamiento de los contenedores y completada la configuración del dominio y del proxy inverso, la aplicación KN-Store se encuentra disponible para los usuarios mediante la dirección web configurada para el entorno de producción.

El acceso a la aplicación se realiza a través del dominio configurado, el cual redirige las solicitudes hacia el servicio correspondiente dentro de la infraestructura desplegada.



6.5 Detener los servicios

En caso de requerir detener los servicios de KN-Store, ejecute el siguiente comando desde la ubicación correspondiente al archivo de configuración:

docker compose -f app.yml down

Este comando detiene y elimina los contenedores creados por Docker Compose.

```
PS C:\Users\APRENDIZ\Documents\Proyecto_KN_S\2026-3311941-projects-grupo-06-knstore\src\main\docker> docker compose -f app.yml down
time="2026-08-12T21:08:12-05:00" level=warning msg="The \\"state\\" variable is not set. Defaulting to a blank string."
[+] Running 3/3
 ✓ Container knstore-app-1      Removed          0.4s
 ✓ Container knstore-mongodb-1  Removed          0.6s
 ✓ Network knstore              Removed          0.4s
PS C:\Users\APRENDIZ\Documents\Proyecto_KN_S\2026-3311941-projects-grupo-06-knstore\src\main\docker> 
```

Este comando detiene y elimina los contenedores creados por Docker Compose.

El volumen:

knstore-mongodb-database mantiene independiente de los contenedores, permitiendo conservar los datos almacenados en MongoDB.

```
PS C:\Users\APRENDIZ\Documents\Proyecto_KN_S\2026-3311941-projects-grupo-06-knstore\src\main\docker> docker volume ls
DRIVER      VOLUME NAME
local       3abb74b17b590af2f6101a647d0b25c6f4cc41c5d2274db2bf8e272bbc27e6b8
local       6b71a4812b4680b16fdc73b1c60651ccbabcf0a9f3eb00f973b1a6fc0854e0e2
local       2097ca0aec4771117ee3c344f91e5c1b774640fe78a99c568326e15b1dfd915b
local       knstore-mongodb-data
```

7. Configuración del servidor AWS

7.1 Creación de la instancia EC2

Para realizar el despliegue del proyecto KN-Store en un ambiente de producción, se utiliza una instancia EC2 de Amazon Web Services (AWS).

La instancia debe utilizar el sistema operativo **Ubuntu Server**, ya que será el entorno donde se instalarán Docker y los demás servicios necesarios para ejecutar la aplicación.

Configuración recomendada

Para la creación de la instancia se deben configurar los siguientes elementos:

Sistema operativo: **Ubuntu Server 22.04 LTS o superior.**

Lanzar una instancia Información

Amazon EC2 le permite crear máquinas virtuales, o instancias, que se ejecutan en la nube de AWS. Comience rápidamente siguiendo los sencillos pasos que se indican a continuación.

Nombre y etiquetas Información

Nombre
knstore-prod [Agregar etiquetas adicionales](#)

▼ Imágenes de aplicaciones y sistemas operativos (Imagen de máquina de Amazon) Información

Una AMI posee el sistema operativo, el servidor de aplicaciones y las aplicaciones de la instancia. Si a continuación no ve una AMI adecuada, utilice el campo de búsqueda o elija [Buscar más AMI](#).

Busque en nuestro catálogo completo que incluye miles de imágenes de sistemas operativos y aplicaciones

Inicio rápido

Amazon Linux macOS **Ubuntu** Windows Red Hat SUSE Linux Debian

Imagen de máquina de Amazon (AMI)

Ubuntu Server 24.04 LTS (HVM), SSD Volume Type
ami-0f8a610b6d1acc0e9 (64 bits (x86)) / ami-09f744a29e9604198 (64 bits (Arm))
Virtualización: hvm Habilitado para ENA: true Tipo de dispositivo raíz: ebs

Configurar almacenamiento Información

1x 50 GiB gp3 Volumen raíz, 3000 IOPS, No cifrado

Almacenamiento:

Configurar un volumen de almacenamiento de **50 GB** para la instancia.

Par de claves

Crear o seleccionar un **Key Pair**, que será utilizado para conectarse posteriormente al servidor mediante SSH.

▼ Par de claves (inicio de sesión) Información

Puede utilizar un par de claves para conectarse de forma segura a la instancia. Asegúrese de que tiene acceso al par de claves seleccionado antes de lanzar la instancia.

Nombre del par de claves - obligatorio

Seleccionar [Crear un nuevo par de claves](#)

Grupo de seguridad

Configurar las reglas de entrada necesarias para permitir la conexión al servidor y el acceso a los servicios de KN-Store.

▼ Configuraciones de red Información

Red Información
vpc-0d11eeb33c31cc366

Subred Información
No hay preferencia (subred predeterminada en cualquier zona de disponibilidad)

Asignación automática de IP pública Información
Habilitar
Se aplican cargos adicionales cuando no se cumplen los límites del nivel gratuito

Firewall (grupos de seguridad) Información

Crear un grupo de seguridad Seleccione un grupo de seguridad existente

Crearemos un nuevo grupo de seguridad denominado "launch-wizard-1" con las siguientes reglas:

☒ Permitir el tráfico de SSH desde Ayuda a establecer conexión con la instancia

Cualquier lugar 0.0.0.0/0

Crear la instancia

Después de revisar la configuración, crear la instancia EC2.

Número de instancias Información
1

Imagen de software (AMI)
Canonical, Ubuntu, 24.04, amd64...más información
ami-0f8e51b66d1acc8e

Tipo de servidor virtual (tipo de instancia)
t3.medium

Firewall (grupo de seguridad)
Nuevo grupo de seguridad

Almacenamiento (volúmenes)
Volúmenes: 1 (50 GiB)

ⓘ Nivel gratuito: Durante el primer año tras abrir una cuenta de AWS, obtiene 750 horas al mes de uso de instancias t2.micro (o t3.micro cuando t2.micro no está disponible) al utilizarlas con AMI del nivel gratuito, 750 horas al mes de uso de direcciones IPv4 públicas, 30 GiB de almacenamiento de EBS, 2 millones de operaciones de E/S, 1 GB de instantáneas y 100 GB de ancho de banda hacia Internet. Los cargos por transferencia de datos no están incluidos como parte de la asignación del nivel gratuito. Es posible que se apliquen cargos en función del estado del nivel gratuito de la cuenta.

Cancelar Lanzar instancia

🔗 Código de versión preliminar

Asignar una IP Elástica

Una vez creada la instancia, asignar una **Elastic IP** para mantener una dirección pública fija. Esta dirección será utilizada posteriormente para configurar el dominio.

Dirección IP elástica asociada

Información

Elija la instancia o la interfaz de red para asociarla a esta dirección IP elástica (54.80.219.209)

Dirección IP elástica: 54.80.219.209

Tipo de recurso

Elija el tipo de recurso al que desea asociar la dirección IP elástica.

Instancia

Interfaz de red

Si asocia una dirección IP elástica a una instancia que ya tiene una dirección IP elástica asociada, la dirección IP elástica asociada anteriormente se desasociará, pero la dirección seguirá asignándose a su cuenta. Más información

Si no se especifica ninguna dirección IP privada, la dirección IP elástica se asociará a la dirección IP privada principal.

Instancia

i-0927015720887e812

Dirección IP privada

La dirección IP privada a la que se asociará la dirección IP elástica.

172.31.22.242

Reasociación

Especifique si la dirección IP elástica se puede volver a asociar a un recurso diferente si ya está asociada a un recurso.

Permitir que se vuelva a asociar esta dirección IP elástica

Cancelar

Asociado

Direcciones IP elásticas (1/1)

Información

Buscar direcciones IP elásticas por atributo o etiqueta

☒	Name	Dirección IPv4 asign...	Tipo
☒	-	54.80.219.209	IP pública

La dirección IP elástica se asignó correctamente.

Dirección IP elástica 54.80.219.209

Direcciones IP elásticas (1/1)

Información

Buscar direcciones IP elásticas por atributo o etiqueta

☒	Name	Dirección IPv4 asign...	Tipo
☒	-	54.80.219.209	IP pública

Puertos configurados en el grupo de seguridad.

Configurar las reglas de entrada necesarias para permitir la conexión al servidor y el acceso a los servicios de KN-Store.

Edit input rules

Information

Inbound rules control the inbound traffic that can reach the instance.

Entry rules

Information

Security group rule ID	Type	Information	Protocol	Port Range	Origin	Information	Description: optional
SGR-0DF3726765A6FE4E7	HTTPS		TCP	443	P...	Q 0.0.0.0/0	
SGR-0AF7054EC303B2SD5	SSH		TCP	22	P...	Q 0.0.0.0/0	
SGR-0E648ABB8C9DAEECF	HTTP		TCP	80	P...	Q 0.0.0.0/0	
-	Custom TCP		TCP	8080	A...	Q 0.0.0.0/0	Backend Spring Boot
-	Custom TCP		TCP	27017	A...	Q 0.0.0.0/0	MongoDB
-	Custom TCP		TCP	3306	A...	Q 0.0.0.0/0	NPM Database
-	Custom TCP		TCP	9000	A...	Q 0.0.0.0/0	Portainer

Add Rule

Rules that originate from 0.0.0.0/0 or ::/0 allow all IP addresses to access the instance. We recommend that you configure security group rules to allow access only from known IP addresses.

Cancel

Preview your changes

Save rules

7.2 Conectarse mediante SSH

Utilizar la clave creada anteriormente y que está en descargas para conectarse a la instancia Ubuntu.

```
PS C:\Users\APRENDIZ> ssh -i "$env:USERPROFILE\Downloads\knstore-key.pem" ubuntu@54.80.219.209
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 7.0.0-1010-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Aug 20 02:00:18 UTC 2026

System load:  0.07               Temperature:   -273.1 C
Usage of /:   21.2% of 47.39GB    Processes:    156
Memory usage: 22%               Users logged in: 0
Swap usage:   0%                IPv4 address for ens5: 172.31.22.242

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

24 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

3 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Wed Aug 19 01:41:45 2026 from 18.206.107.29
ubuntu@ip-172-31-22-242:~$
```

Actualizar Ubuntu

Una vez conectado al servidor, actualizar los paquetes del sistema:

sudo apt update

```
Last login: Wed Aug 19 01:41:45 2026 from 18.206.107.29
ubuntu@ip-172-31-22-242:~$ sudo apt update
Get:1 https://download.docker.com/linux/ubuntu noble InRelease [48.5 kB]
Hit:2 https://deb.nodesource.com/node_20.x nodistro InRelease
Get:3 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:4 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [64.1 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [952 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [46.4 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1201 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [76.2 kB]
Hit:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
```

sudo apt upgrade -y

```
ubuntu@ip-172-31-22-242:~$ sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
Get more security updates through Ubuntu Pro with 'esm-apps' enabled:
```

Instalar Docker

Finalmente, instalar Docker en la instancia y comprobar que funciona correctamente:

sudo apt install -y docker.io docker-compose-plugin

```
ubuntu@ip-172-31-22-242:~$ sudo apt install -y docker.io docker-compose-plugin
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

Verificar que Docker se haya instalado correctamente:

docker --version

```
ubuntu@ip-172-31-22-242:~$ docker --version
Docker version 29.7.1, build e9452d6
```

Verificar Docker Compose:

docker compose version

```
ubuntu@ip-172-31-22-242:~$ docker compose version
Docker Compose version v5.4.0
```

Finalmente, comprobar que el servicio de Docker esté activo:

sudo systemctl status docker

Debe aparecer:

active (running)

```
ubuntu@ip-172-31-22-242:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-08-20 00:57:42 UTC; 1h 6min ago
     TriggeredBy: ● docker.socket
      Docs: https://docs.docker.com
    Main PID: 789 (dockerd)
      Tasks: 123
     Memory: 164.6M (peak: 167.4M)
        CPU: 9.784s
    CGroup: /system.slice/docker.service
            └─ 789 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
              1425 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8000 -container-ip 172.18.0.2
              1443 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 8000 -container-ip 172.18.0.2
              1463 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 9000 -container-ip 172.18.0.2
              1474 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 9000 -container-ip 172.18.0.2
              1501 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 3000 -container-ip 172.18.0.2
              1507 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 3000 -container-ip 172.18.0.2
              1530 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8080 -container-ip 172.18.0.2
              1537 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 8080 -container-ip 172.18.0.2
              1578 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 80 -container-ip 172.18.0.2
              1587 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 80 -container-ip 172.18.0.2
              1604 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 81 -container-ip 172.18.0.2
              1611 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 81 -container-ip 172.18.0.2
              1627 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 443 -container-ip 172.18.0.2
              1632 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 443 -container-ip 172.18.0.2

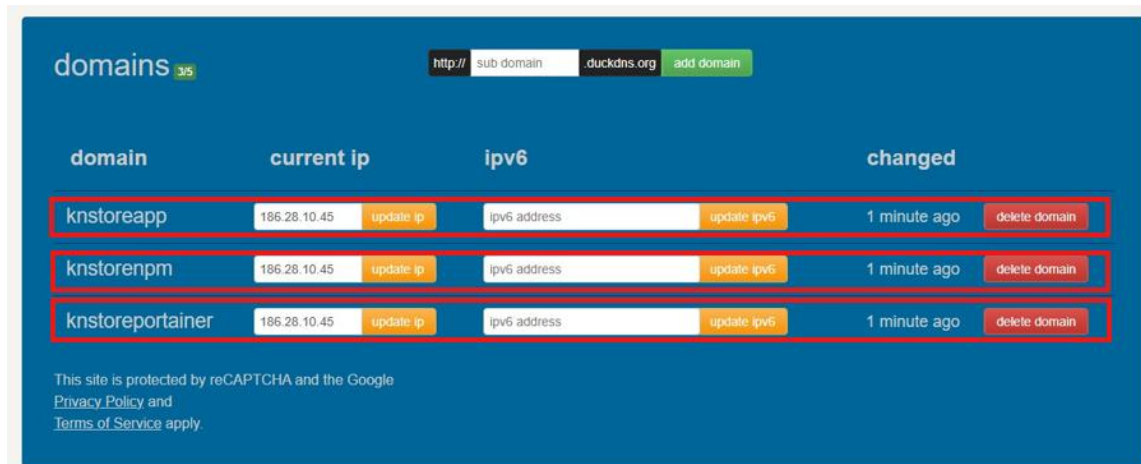
Aug 20 00:57:41 ip-172-31-22-242 dockerd[789]: time="2026-08-20T00:57:41.447943622Z" level=info msg=">
Aug 20 00:57:41 ip-172-31-22-242 dockerd[789]: time="2026-08-20T00:57:41.463786016Z" level=info msg=">
Aug 20 00:57:41 ip-172-31-22-242 dockerd[789]: time="2026-08-20T00:57:41.478386386Z" level=info msg=">
Aug 20 00:57:41 ip-172-31-22-242 dockerd[789]: time="2026-08-20T00:57:41.793340168Z" level=info msg=">
Aug 20 00:57:41 ip-172-31-22-242 dockerd[789]: time="2026-08-20T00:57:41.818695115Z" level=info msg=">
```

8. Configuración del DNS

Para configurar el acceso mediante dominio, se utilizó Duck DNS como servicio de DNS dinámico. En el panel de administración se crearon los subdominios necesarios para los diferentes servicios del proyecto.

En la configuración mostrada se encuentran los siguientes dominios:

- **knstoreapp.duckdns.org:** utilizado para la aplicación KN-Store.
- **knstorenpm.duckdns.org:** utilizado para Nginx Proxy Manager.
- **knstoreportainer.duckdns.org:** utilizado para Portainer.



Se accedió al panel de Duck DNS y se crearon los subdominios correspondientes a la aplicación y a las herramientas utilizadas durante el despliegue. Cada subdominio se configuró para apuntar a la dirección IP pública del servidor AWS.

9. Dominio y SSL

9.1 Configuración del Administrador de proxy Nginx

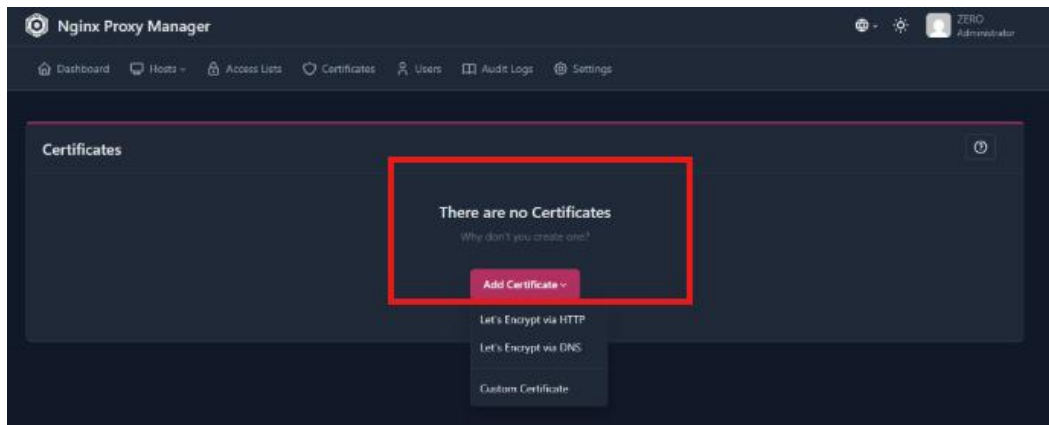
Descripción

Para permitir el acceso a KN-Store mediante un dominio público y configurar una conexión segura mediante HTTPS, se utilizó Nginx Proxy Manager (NPM) como servidor proxy inverso.

Dominio utilizado: **knstoreapp.duckdns.org**

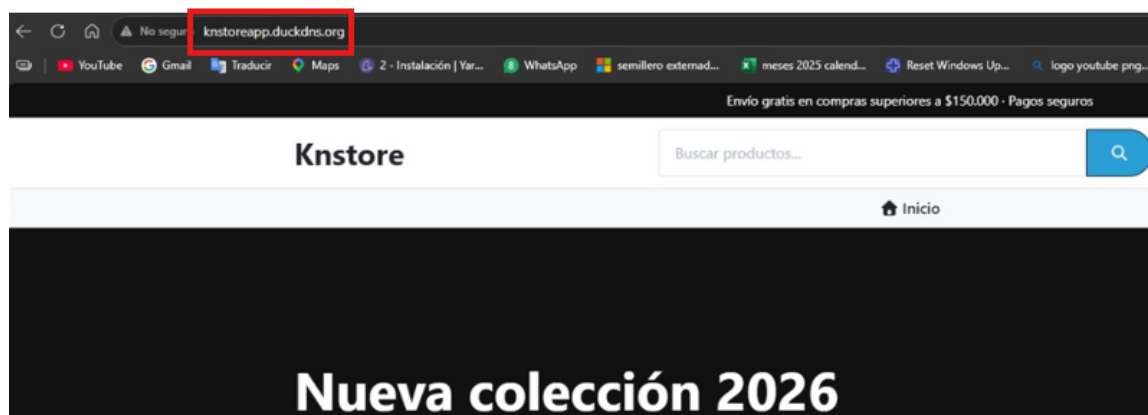
9.2 Acceso a Nginx Proxy Manager

Una vez instalado y ejecutándose Nginx Proxy Manager en la instancia AWS, se accedió al panel administrativo mediante donde se gestionan los Proxy Hosts, certificados SSL y demás opciones.



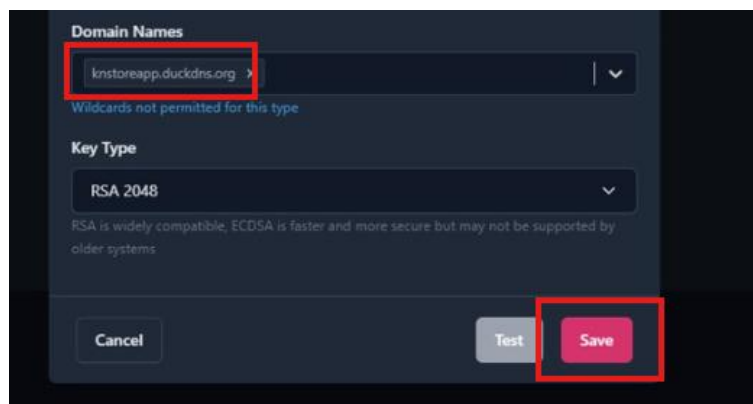
9.3 Configuración del Proxy Host

Se seleccionó Add Proxy Host dentro de Proxy Hosts. Se configuraron el dominio knstoreapp.duckdns.org, el esquema HTTP, el destino knstore-backend y el puerto 8080. El acceso se estableció como públicamente accesible.



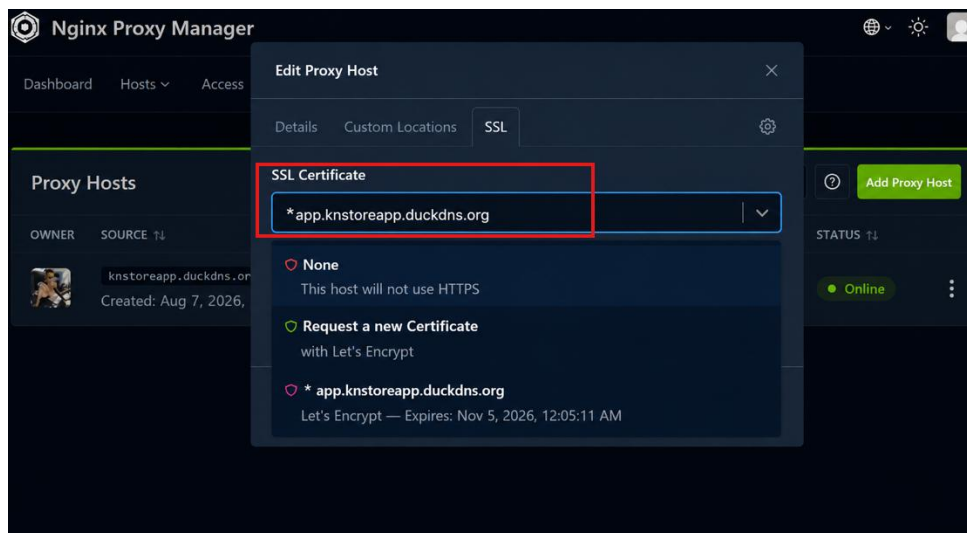
9.4 Creación del Proxy Host

Después de guardar la configuración, Nginx Proxy Manager muestra el dominio registrado, el destino http://knstore-backend:8080 y el estado Online.



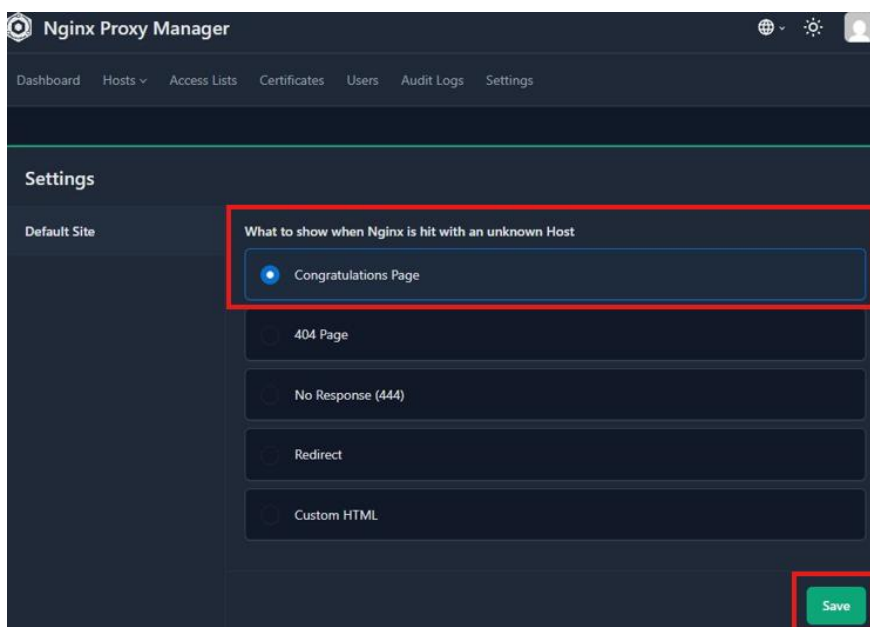
9.5 Configuración del certificado SSL

Desde la sección Certificates de Nginx Proxy Manager se realizó la configuración del certificado de seguridad mediante Let's Encrypt. Para el dominio de la aplicación se utilizó el certificado identificado como ***app.knstoreapp.duckdns.org**, el cual posteriormente se seleccionó desde la configuración SSL del Proxy Host. La configuración permite gestionar el acceso a la aplicación mediante HTTPS, proporcionando una conexión cifrada entre el usuario y el servidor.



9.6 Configuración general de Nginx Proxy Manager

En Settings → Default Site se puede establecer el comportamiento de Nginx cuando recibe una solicitud cuyo dominio no coincide con un Proxy Host configurado. Se mantuvo la configuración predeterminada.

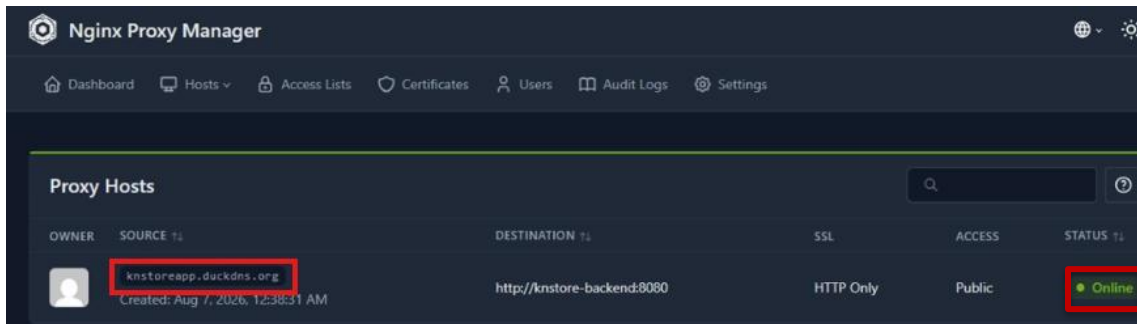


9.7 Resultado de la configuración

El flujo de acceso queda establecido de la siguiente manera:

Usuario → app.knstoreapp.duckdns.org → Nginx Proxy Manager → knstore-backend:8080 → Backend KN-Store → MongoDB

Aquí puedes colocar la captura donde aparece el **Proxy Host en estado Online**.



10. Resumen del Manual de Instalación

El presente manual describe el proceso necesario para preparar, instalar, configurar y ejecutar el proyecto **KN-Store** en un ambiente de producción.

Inicialmente, se presentan los requisitos del servidor y las herramientas necesarias para el despliegue, incluyendo **Docker y Docker Compose**. Posteriormente, se explica la configuración de los contenedores de la aplicación y de MongoDB mediante Docker Compose.

También se documenta la configuración de los servicios mediante Docker, incluyendo la ejecución de la aplicación y la base de datos MongoDB. Para el despliegue en un servidor, se describe la creación y preparación de una instancia Ubuntu en AWS, la conexión mediante SSH y la instalación de Docker.

Finalmente, se presenta la configuración del dominio, Nginx Proxy Manager y el certificado SSL, permitiendo establecer un mecanismo de acceso a la aplicación mediante un dominio configurado y gestionar las conexiones hacia los servicios de KN-Store.

De esta manera, el manual reúne los principales pasos necesarios para reproducir el proceso de instalación y configuración del proyecto.

11. Conclusiones

La elaboración de este manual permitió documentar de manera organizada el proceso de instalación y configuración del proyecto **KN-Store**, identificando las herramientas, configuraciones y servicios necesarios para su correcto funcionamiento.

La utilización de tecnologías como **Spring Boot, React y MongoDB** permite estructurar el proyecto mediante diferentes componentes, mientras que Docker facilita la administración y ejecución de los servicios requeridos por la aplicación.

Por otra parte, el uso de una instancia en **AWS**, junto con la configuración de un dominio y Nginx Proxy Manager, permite disponer de una infraestructura para realizar el despliegue y gestionar el acceso hacia la aplicación.

Finalmente, contar con un manual de instalación facilita la reproducción del entorno del proyecto, sirve como guía para desarrolladores o personal técnico y permite conservar documentado el procedimiento utilizado durante la implementación de **KN-Store**.